

End-to-End Autonomous Driving Algorithm Based on PPO and Its Implementation

Jianfeng Zhao, Yang Zhao, Weixia Li, Chaoqun Zeng

1. School of Automotive and Transportation Engineering, Shenzhen Polytechnic University, Shenzhen 518000, P. R. China
E-mail: zjf@szpu.edu.cn, yzhao83-c@my.cityu.edu.hk, WeixiaLi@szpu.edu.cn, zengchaoqun@szpu.edu.cn

Abstract: With the commercialization of vehicles equipped with partial autonomous driving capabilities, achieving fully autonomous driving remains a hot topic among researchers. In contrast to modular autonomous vehicle control, end-to-end autonomous driving aims to replicate human driving by directly translating sensor inputs into vehicle control outputs, making it the ultimate goal in autonomous driving. In this study, we propose an end-to-end autonomous driving algorithm based on the Proximal Policy Optimization (PPO) reinforcement learning algorithm and verify its feasibility. Firstly, we construct an autonomous driving algorithm based on the PPO algorithm, where pre-encoded images and vehicle state information are input to a policy based on deep neural networks, directly generating speed and steering controls for the vehicle, thus achieving true end-to-end autonomous driving. Furthermore, using the CARLA simulation environment for autonomous vehicles, we define a reward mechanism for driving behavior and establish a reinforcement learning environment to train and validate the proposed algorithm. Finally, we analyze the algorithm's feasibility based on rewards, cumulative average rewards, average episode length, and average driving distance. While the algorithm still has a long way to go before practical application, it lays the foundation for further implementation of end-to-end autonomous driving algorithms.

Key Words: End-to-End Autonomous Driving, Proximal Policy Optimization (PPO), Deep Reinforcement Learning, CARLA Simulation Environment

1 Introduction

Autonomous driving vehicles are currently a research hotspot, and the existing applicable autonomous driving algorithms are all implemented using a modular structure. In contrast to modular solutions, end-to-end only requires training one model, significantly simplifying the workload, while the shared backbone network enhances computational efficiency. Moreover, end-to-end models resemble human thought processes more closely compared to multi-module solutions, offering better generalization capabilities and to some extent alleviating long-tail issues. However, end-to-end autonomous driving still faces many challenges in industrial applications, such as numerous and multimodal inputs, high precision and speed requirements for computations, and multiple possible outputs. Figure 1 illustrates the difference between the two autonomous driving approaches.

In recent years, researchers have made remarkable strides in developing advanced algorithms and frameworks to enhance the capabilities of autonomous driving systems. This paper presents a comprehensive overview of recent advancements in autonomous driving research, focusing on a range of innovative approaches and methodologies.

Scheel et al. [1] introduced an offline policy gradient method for learning complex urban driving strategies from large-scale real-world demonstrations, offering insights into effective learning strategies for navigating

This work is supported by the Key research project of Guangdong Provincial Department of Education under Grant 2021ZDZX1140, the Guangdong Basic and Applied Basic Research Foundation under Grant 2021A1515110804, the Scientific Research Startup Fund for Shenzhen High-caliber Personnel of SZPT under Grant 6022310045k, the Shenzhen Polytechnic University Research Fund under Grant 6024310009K.

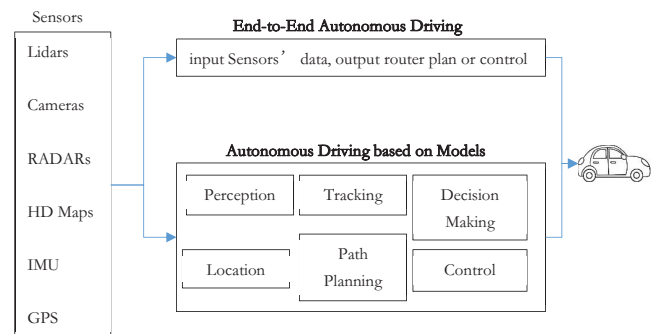


Fig. 1: A comparison of two different autonomous driving implementation modes

urban environments.

Wu et al. [2] proposed PPGeo (Policy Pre-training based on Geometric Modeling), a self-supervised framework for policy pre-training in visual motion driving, aiming to improve the adaptability and robustness of autonomous driving systems.

Hossain et al. [3] applied Deep Q-learning to autonomous driving, demonstrating the feasibility of the algorithm through validation in the CARLA simulator, highlighting its potential for enhancing autonomous driving capabilities.

Inspired by the UNet architecture, researchers in [4] proposed a lightweight UNet using depthwise separable convolutions (DSUNet) for end-to-end learning of lane detection and path prediction in autonomous driving, offering an efficient solution for real-time processing tasks.

Furthermore, authors in [5] present HIIIL (Hierarchical Interpretable Imitation Learning), a two-stage end-to-end autonomous driving model designed for complex urban scenarios, integrating interpretable BEV mask

and steering angle for improved interpretability and performance.

Lastly, researchers in [6] proposed an interpretable deep reinforcement learning method for end-to-end autonomous driving, addressing the need for transparent decision-making processes in autonomous vehicles.

Considering multimodality, [7] explored early, mid, and late fusion schemes in both multisensory and single-sensor settings, offering insights into leveraging multiple modalities for enhanced perception and decision-making in autonomous driving.

A comprehensive analysis of over 250 papers can be found in the review article [8]. It serves as a valuable summary of the recent advancements in end-to-end autonomous driving and provides further references for interested readers.

Compared to previous research, the core of this paper lies in the application of the PPO (Proximal Policy Optimization) algorithm, which is one of the most important reinforcement learning algorithms, to the field of autonomous driving to achieve end-to-end autonomous driving. In terms of input, this paper only takes data captured from a single camera and the current state of the vehicle. In terms of output, the policy model directly outputs the vehicle's longitudinal and lateral control commands, making it a typical end-to-end model. The training of the model adopts reinforcement learning algorithms, eliminating the need for manual intervention and simplifying the training process. Different from existing research results, the main contributions of this paper are as follows:

- Completely end-to-end: The proposed model takes camera-captured data as input and outputs the vehicle's control commands. Unlike step-by-step end-to-end algorithms like "Planning-oriented Autonomous Driving," the algorithm proposed in this paper directly provides acceleration and steering control commands based on data from the vehicle's camera.
- Exploration of the application of reinforcement learning algorithms in end-to-end autonomous driving algorithms and the introduction of a reward evaluation method for reinforcement learning algorithms.
- In order to reduce computational complexity and meet the real-time requirements of vehicle driving, pre-training with VAE (Variational Autoencoder) is used for image recognition, improving the training speed and vehicle driving recognition speed.

2 Modeling of End-to-End Autonomous Driving Algorithm Based on PPO

2.1 General Model of Reinforcement Learning Algorithm

In the framework of reinforcement learning, an agent receives a state S_t from the environment at each time step t , selects an action A_t based on the current policy π , and then the environment transitions to a new state s_{t+1} and provides an immediate reward r_{t+1} . The agent's objective is to choose actions to maximize the

future cumulative reward. It can be mathematically expressed as follows:

- 1) State space S : Defines all possible environmental states.
- 2) Action space A : Defines all actions that the agent can choose.
- 3) Policy π : The policy is a mapping from states to actions, defined as $\pi(a|s)$, representing the probability of choosing action a in state s .
- 4) Reward function R : The reward function $R(s, a, s')$ defines the immediate reward obtained by the agent when taking action a in state s and transitioning to state (s') .
- 5) State transition probability P : The state transition probability $P(s'|s, a)$ defines the probability of transitioning to state s' after taking action a in state s .
- 6) Cumulative reward G_t : Represents the reward that the agent can obtain from interacting with the environment, and its expression is:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where γ is the discount factor $\gamma \in [0, 1]$, used to calculate the present value of future rewards, reflecting the importance of future rewards.

The goal of reinforcement learning is to find the optimal policy π^* to maximize the cumulative reward G_t . This is usually achieved by learning a value function, which measures the expected cumulative reward obtained by following a specific policy in a given state. There are two forms of the value function: the state value function $V^\pi(s)$ and the action value function $Q^\pi(s, a)$.

- State value function $V^\pi(s)$: Represents the expected return that can be obtained by following the policy π in state s .

$$V^\pi(s) = E[G_t | S_t = s]$$

- Action value function $Q^\pi(s, a)$: Represents the expected return that can be obtained by taking action a in state s and then following the policy π .

$$Q^\pi(s, a) = E[G_t | S_t = s, A_t = a]$$

Define $(A_\pi(s, a))$ as the advantage due to taking action a :

$$A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s) \quad (1)$$

The core of reinforcement learning lies in continuously iterating to update the policy and value function, seeking an optimal policy π_* , such that for $\forall \pi$:

$$v_*(s) \geq v(s) \quad (2)$$

Common reinforcement learning algorithms include Q-learning, SARSA, Deep Q-Network (DQN), and policy gradient algorithms such as Policy Gradients.

2.2 Proximal Policy Optimization Algorithm

2.2.1 The mathematic model of PPO

PPO (Proximal Policy Optimization) is a widely used algorithm in reinforcement learning proposed by OpenAI. Compared to policy gradient algorithms, PPO calculates the probability ratio between the new and old policies, and then clips this ratio to limit the magnitude of policy updates and maintain stability, avoiding performance collapse caused by excessive policy updates. The formula for clipping the probability ratio is:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

In PPO (Proximal Policy Optimization), the objective function consists of two components: clipped advantage function and entropy bonus. The advantage function A_t measures the relative value of an action a_t compared to the average action.

The clipped advantage function is given by:

$$L^{CLIP}(\theta) = \hat{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (3)$$

Here, $\hat{E}_t$ denotes the expectation over time step t , $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)$ limits the ratio $r_t(\theta)$ within the range $[1 - \epsilon, 1 + \epsilon]$, and ϵ is a small positive number (e.g., 0.1 or 0.2) controlling the degree of clipping.

To encourage exploration in the policy over a wider state space, PPO usually incorporates an entropy term in the objective function:

$$L^{TOTAL}(\theta) = L^{CLIP}(\theta) + c_1 L^{VF}(\theta) - c_2 S\pi_\theta \quad (4)$$

Here, $L^{VF}(\theta)$ represents the loss of the value function, $S\pi_\theta$ is the entropy of the policy π_θ at state s_t , and c_1 and c_2 are weight coefficients for balancing the contributions from different parts.

2.2.2 Steps of the PPO algorithm

- 1) Data Collection: Interact with the environment using the current policy $\pi_{\theta_{old}}$ to gather a sequence of states, actions, and rewards.
- 2) Estimate Advantage Function: Compute the advantage function A_t for each time step.
- 3) Optimize PPO Objective Function: Update the policy parameters θ through gradient ascent to maximize $L^{TOTAL}(\theta)$.
- 4) Repeat: Replace the old policy with the updated policy and repeat the above steps until a termination condition is met.

2.3 End-to-End Autonomous Driving Algorithm Based on PPO

2.3.1 The framework of algorithm

The overall framework of the algorithm is shown in figure 2. The basic idea of the algorithm is to collect image data from the vehicle's camera during operation, input the image data into a pre-trained image encoder (Variable Autoencoder, VAE), and then input

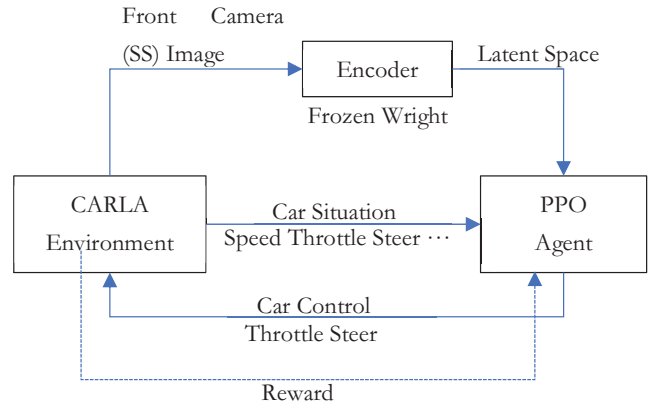


Fig. 2: Overall Framework of the Algorithm

the encoding result of the encoder into the Agent, which outputs the decision results. Therefore, the algorithm includes two independent neural networks, namely the image encoder and the Agent. The image encoder takes the image data captured by the vehicle's camera as input and outputs the encoding of the image. The Agent uses the Actor-Critic model, taking the encoding output by the VAE as input and outputting the control decisions of the vehicle, steering and acceleration. Its input and output are shown in figure 3.

The basic idea of the algorithm is to interact with the environment to obtain (S_t, A_t, S_{t+1}, R_t) , and update the policy every 10 episodes. S_t is the state of the environment, which is a frame of image data captured by the vehicle's camera. A_t is the action taken by the Agent, which is a continuous value that includes the throttle and steering commands issued to the vehicle. S_{t+1} is the next frame of image data collected by the vehicle's camera when A_t is taken under S_t . The reward is evaluated according to the vehicle's operating status, which will be detailed in the following section.

2.3.2 Evaluation Method for Rewards

- 1) If a collision occurs or the deviation from the desired path exceeds a certain threshold (set to 3 meters), the reward is -10 and the game ends.
- 2) If the running time exceeds 10 seconds but the vehicle speed is less than 1, the reward is -10 and the game ends.
- 3) If the vehicle speed exceeds the maximum allowed speed (set to 25 m/s), the reward is -10 and the game ends.
- 4) If the vehicle speed is below the minimum required speed, the reward is calculated as $(\text{vehiclespeed}/\text{minimumspeed}) \times \text{lanecenterdeviation} \times \text{headingdeviation}$. Note that the lane center deviation is calculated as 1 minus the ratio of the current deviation to the maximum allowed deviation, and the heading deviation is calculated as 1 minus the ratio of the current angle to the maximum allowed angle.
- 5) If the vehicle speed is greater than the target speed, the reward is calculated as $(1 - (\text{vehiclespeed} - \text{targetspeed})/(\text{maximumspeed} - \text{targetspeed})) \times$

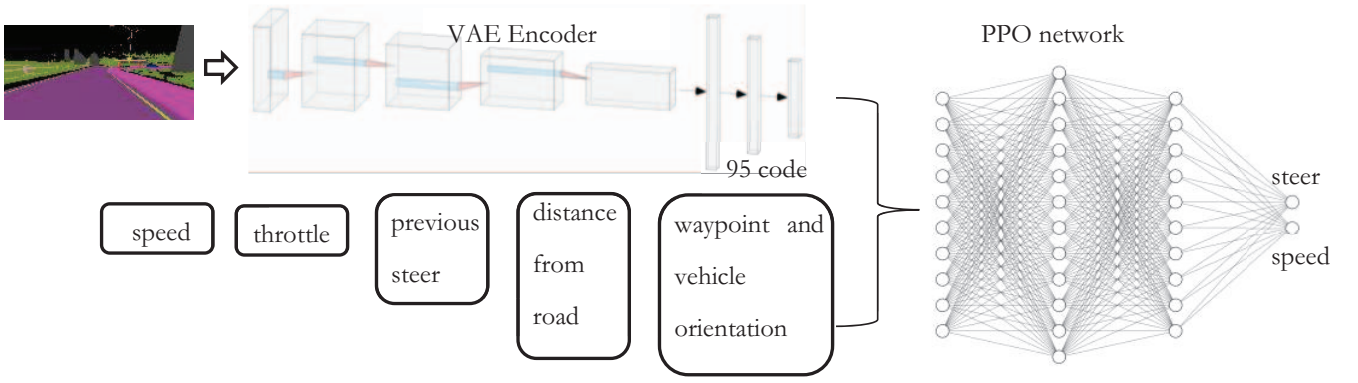


Fig. 3: Main Networks and their Inputs and Outputs

Algorithm 1 Pseudocode for End-to-End Autonomous Driving Algorithm based on PPO

Require: Total time steps, $total_timestep$;
 Ensure: Trained agent model;
 1: Initialize the testing environment;
 2: while $timestep < total_timestep$ do
 3: Initialize episode;
 4: for t in $episode_length$ do
 5: Encode the observation from the front camera;
 6: Agent gets action A_t according to the observation S_t ;
 7: Input the action A_t into the environment to get the next observation S_{t+1} , reward R_t , and check if the episode is terminal, $Done$;
 8: Put $(S_t, A_t, S_{t+1}, R_t, Done)$ into the memory pool;
 9: if $Done$ then
 10: $episode++ = 1$;
 11: break;
 12: end if
 13: end for
 14: if $episode \bmod 10 == 0$ then
 15: Agent learns;
 16: end if
 17: end while return Trained agent model;

lanecenterdeviation $\times$ headingdeviation.

- 6) If the vehicle speed is within the desired range, the reward is calculated as lanecenterdeviation $\times$ headingdeviation.

In this paper, the Agent uses the Actor-Critic approach. For an Actor-Critic, corresponding to the calculation formula mentioned in section 2.1, the update algorithm is as follows:

The PPO update algorithm, which is the method for calculating the error of neural network weight updates, uses the calculation formula from section 2.2.1. The loss calculation includes three parts:

- Value function loss $L^{VF}(\theta)$: Uses mean squared error (MSE) as the loss function. By calculating the mean squared error between the value prediction for each state output by the policy network Critic and the discounted cumulative reward (rewards), this part of the loss reflects the difference between the current state prediction and the actual reward. By inputting this loss into the training network,

Algorithm 2 Pseudocode for Policy Gradient Optimization with Clipped Objective

Require: $n_updates_per_iteration$: Number of updates per iteration; ϵ : Clip parameter; c_1, c_2 : Coefficients for value function loss and entropy term;
 1: for $_$ in $n_updates_per_iteration$ do
 2: Obtain logprobs, values, dist_entropy by policy evaluation;
 3: Calculate the policy advantage $r_t(\theta)$;
 4: $ratios = logprobs - old_logprobs$;
 5: $advantages = rewards - values$;
 6: Clip the objective function 3:
 7: $L^{CLIP}(\theta) = \hat{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$;
 8: Calculate the total loss function 4:
 9: $L^{TOTAL}(\theta) = L^{CLIP}(\theta) + c_1 L^{VF}(\theta) - c_2 S\pi_\theta$;
 10: Take policy gradient optimization using loss;
 11: end for
 12: Update policy;
 13: Clear memory;

Critic's prediction for the current state becomes more accurate over time.

- Entropy of the policy π_θ in state s_t $S(\pi_\theta)$: Obtained through the evaluation function evaluate of the policy network Critic, it represents the entropy of the probability distribution corresponding to the current policy. The entropy of the policy is used to encourage exploration and avoid premature convergence to a local optimal solution by increasing the randomness of the policy.
- Clipped advantage function $L^{CLIP}(\theta)$: According to the policy gradient theorem, the gradient of policy updates is proportional to the logarithm of policy updates. Therefore, the $\log(\pi) * advantage$ is used for policy updates, i.e., calculating the Actor network loss. Unlike directly applying the updates to the policy, PPO uses clipping to ensure the stability of policy updates. The calculation formula is shown as

$$L^{CLIP}(\theta) = \hat{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (5)$$

3 End-to-end autonomous driving algorithm based on PPO

3.1 VAE model for autonomous driving

Considering the computational complexity and the possibility of network convergence (to ensure network convergence), the input image is first encoded, and an autoencoder VAE model for autonomous driving is established.

As show in figure 3, there are a total of 4 convolutional layers (Conv), and the results of the convolution are sent to a linear layer 9216×1024 (Gemm). Then, two linear layers 1024×95 are used to generate two encoding sampling functions μ and σ respectively from the linear layer. The sampling encoding is obtained by multiplying and adding the μ and σ obtained from the normal distribution function (black part). Finally, the encoding is normalized using the Tanh function.

3.2 PPO algorithm model

The PPO algorithm has a similar network structure for both Critic and Actor, with the main body consisting of 4 neural network units, each containing a linear layer, a normalization layer, and a Tanh layer. The dimensions of each layer expand from the observation dimension of 100 to 1000, then decrease to 500, 300, 100, and finally to the output dimension. Note that in the figure, only the linear layer is shown for convenience, without showing the normalization layer and Tanh layer. The output dimension of the Actor structure is the action dimension, which in this article is the throttle and steering actions, while the Critic only outputs a single evaluation, which is an estimate of future rewards.

4 Experimental Results

4.1 Experimental Platform

The hardware configuration used in the experiments in this article is shown in table 1.

Table 1: Hardware configuration used in the experiment

Hardware Parameters	Specifications
CPU	11th Gen Intel® Core™ i9-11900F @ 2.50GHz
CPU number	16
Memory	64GB
GPU	NVIDIA DMI
GPU Driver Version	545.23.06
CUDA Version	12.3

The operating system is Ubuntu 20.04, and PyTorch is selected as the deep learning framework. The specific software environment configuration is shown in table 2.

Among them, the Carla version is 0.9.13, as shown in table 3.

4.2 Experimental Parameter Settings

There many parameters in the experiment. Table 4 shows the values and the meaning of the parameters.

Table 2: Software version used in the system

Software	Version
future	0.18.3
numpy	1.21.1
poetry	1.3.2
poetry-core	1.4.0
poetry-plugin-export	1.3.1
pygame	2.1.2
tensorboard	2.11.2
tensorboard-data-server	0.6.1
tensorboard-plugin-wit	1.8.1

Table 3: Carla version used in the system

Name	Parameters
Name	Carla-simulator
Version	0.9.13
Architecture	amd64
Description	Open source simulator for automotive driving research

4.3 Experimental Results

The proposed end-to-end autonomous driving algorithm based on PPO was tested in the Carla environment. The testing environment in Carla utilized Town7, as depicted in the figure 4.



Fig. 4: Carla training environment and third-person view of vehicle operation

Multiple experiments demonstrated that with increasing training iterations, the algorithm's return steadily increased, indicating its convergence and feasibility. Taking one specific experimental result as an example, as shown in the figure 5, the algorithm's return consistently improved with training time. At Episode 2068, Timestep 1730403, the return reached 663.98, with a historical cumulative average return of 426.94.

To further observe the algorithm's operation, analysis of the running process was conducted, as illustrated in the figure 5.

The figure 5 depicts the average return for each episode as the algorithm continuously interacts with the environment. It is evident from the graph that the algorithm's average reward continues to increase, confirming its convergence.

In contrast to the average episode reward, the figure 6 displays the reward for every 5 episodes. While there is some fluctuation in the return for each episode, the overall trend demonstrates consistent improvement, confirming the algorithm's convergence. It is worth noting that the dashed line in the graph represents a quadratic fit to clearly display the trend of the algorithm's reward.

The figure 7 shows the average length for every 5 episodes, reflecting the algorithm's ability to control

Table 4: Experimental parameter settings

1	BACH_SIZE	1	The number of samples used in each parameter update step
2	IM_WIDTH	160	The width (in pixels) of the images captured by the onboard camera
3	IM_HEIGHT	80	The height (in pixels) of the images captured by the onboard camera
4	GAMMA	0.99	The discount factor for reinforcement learning rewards
5	MEMORY_SIZE	5000	The size of the buffer for reinforcement learning
6	EPISODES	1000	The maximum number of EPISODES allowed for each learning iteration
7	EPISODE_LENGTH	75000	The maximum length allowed for each EPISODE
8	TOTAL_TIMESTEPS	2e10	The maximum time steps allowed for the entire training process
9	ACTION_STD_INIT	0.2	The initial standard deviation of actions used in the action sampling function
10	ACTION_STD_DECAY_RATE	0.05	The rate at which the standard deviation of actions decreases
11	MIN_ACTION_STD	0.05	The minimum standard deviation of actions
12	ACTION_STD_DECAY_FREQ	5e5	The frequency (in time steps) at which the standard deviation of actions decreases
13	LEARNING_RATE	1e-6	The learning rate for reinforcement learning
14	POLICY_CLIP	0.2	The allowed range for PPO policy clipping

Table 5: Simulation Parameters

Name	Value
Episode	2068
Timestep	1730403
Time_length	17.621811
Reward	663.98
Average Reward	426.94

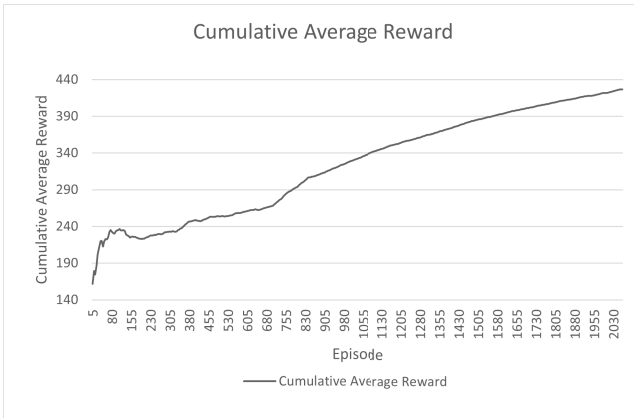


Fig. 5: Average cumulative return

the vehicle's duration without collision or lane deviation. As training progresses, the controlled duration consistently increases. Similarly, a quadratic fit is used to display the trend, as indicated by the dashed line in the graph.

The figure 8 displays the distance traveled by the vehicle for every 5 episodes, reflecting the distance covered under the algorithm's control. It is apparent that the distance traveled steadily increases with the progression of algorithm training, eventually reaching stability. From the graph, it can be concluded that the algorithm achieves convergence after a period of operation.

At a micro-level, the distance from the center lane reflects the precision of vehicle control. The graph 9 shows the average distance from the center for every 5 episodes. It can be observed that the average deviation decreased from 0.65 initially to 0.45, indicating increas-

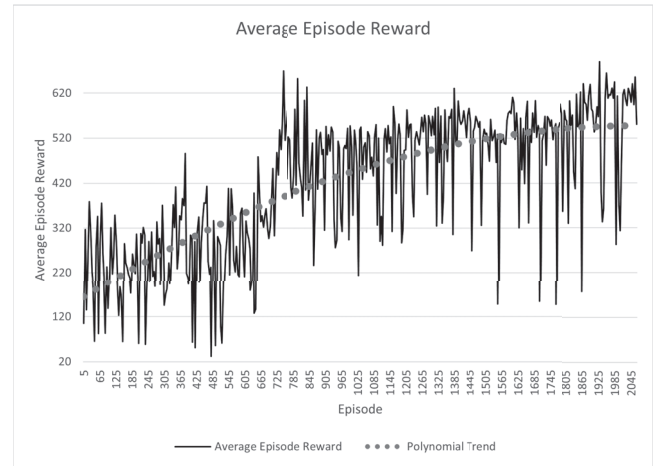


Fig. 6: Return for each episode

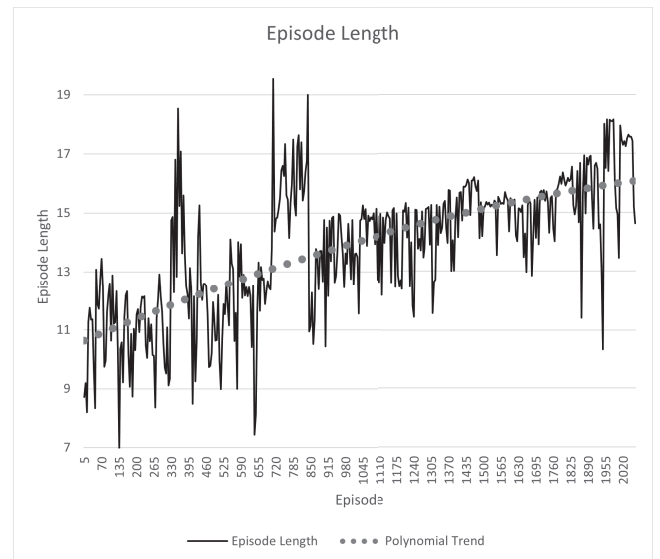


Fig. 7: Length of each episode

ingly precise control as the algorithm is trained. This reflects the feasibility of the algorithm's training.

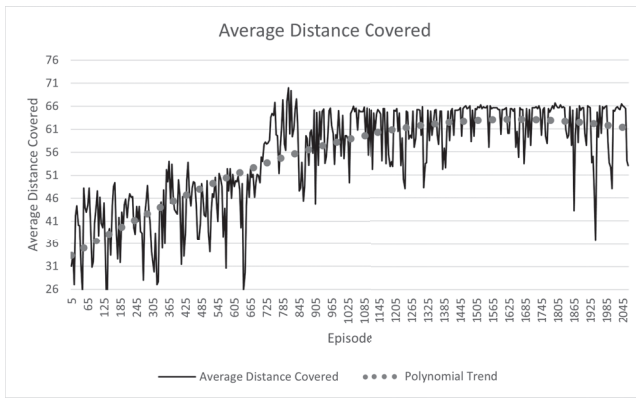


Fig. 8: Distance traveled by the vehicle

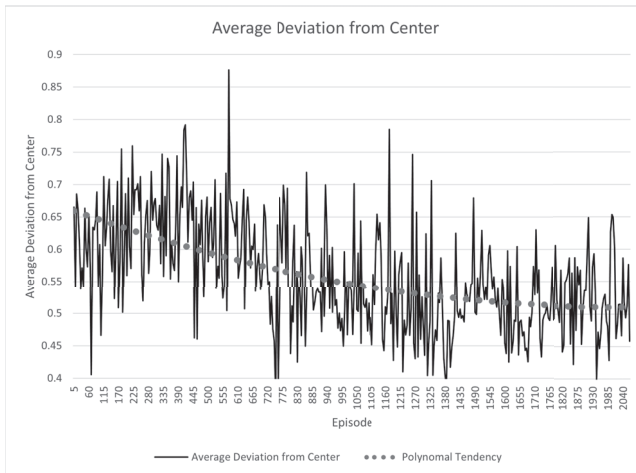


Fig. 9: Distance from lane center

5 Conclusion

This paper presents an end-to-end autonomous driving algorithm based on the reinforcement learning algorithm PPO. The algorithm takes data collected by the in-vehicle camera as input and outputs vehicle control, representing a true end-to-end autonomous driving algorithm. Testing was conducted on the Carla platform, and through the observation of metrics such as the reward for each training session, average cumulative reward, average control duration, average driving distance, and average deviation from lane points, the feasibility of the algorithm was verified, confirming its convergence. Although the PPO network has achieved convergence and verified the feasibility of the algorithm, its practical application still faces challenges. In the current Carla environment, the trained strategy can only maintain autonomous driving for a few tens of seconds before deviating from the intended trajectory. Therefore, while validating the feasibility of the algorithm presented in this paper, further optimization of the algorithm is necessary to enhance its autonomous driving capability. Potential research directions for further study include: 1. Introducing time series data. The current network only takes single images as input, which makes it difficult to predict the vehicle's next state. Thus, introducing time series data is the first direction to explore in the next steps. 2. Optimizing

the reward evaluation mechanism; 3. Introducing data from multiple cameras; 4. Using more advanced encoding models; 5. Increasing the depth of the policy model and utilizing more advanced policy models.

References

- [1] O. Scheel, L. Bergamini, M. Wolczyk, B. Osinski, and P. Ondruska, "Urban driver: Learning to drive from real-world demonstrations using policy gradients," in 5th Annual Conference on Robot Learning, 2021.
- [2] P. Wu, L. Chen, H. Li, X. Jia, J. Yan, and Y. Qiao, "Policy pre-training for autonomous driving via self-supervised geometric modeling," in International Conference on Learning Representations, 2023.
- [3] J. Hossain, "Autonomous driving with deep reinforcement learning in carla simulation," 2023.
- [4] D.-H. Lee and J.-L. Liu, "End-to-end deep learning of lane detection and path prediction for real-time autonomous driving," *SIGNAL IMAGE AND VIDEO PROCESSING*, vol. 17, pp. 199–205, FEB 2023.
- [5] S. Teng, L. Chen, Y. Ai, Y. Zhou, Z. Xuanyuan, and X. Hu, "Hierarchical interpretable imitation learning for end-to-end autonomous driving," *IEEE TRANSACTIONS ON INTELLIGENT VEHICLES*, vol. 8, pp. 673–683, JAN 2023.
- [6] J. Chen, S. E. Li, and M. Tomizuka, "Interpretable end-to-end urban autonomous driving with latent deep reinforcement learning," *IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS*, vol. 23, pp. 5068–5078, JUN 2022.
- [7] Y. Xiao, F. Codevilla, A. Gurram, O. Urfalioglu, and A. M. Lopez, "Multimodal end-to-end autonomous driving," *IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS*, vol. 23, pp. 537–547, JAN 2022.
- [8] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, "End-to-end autonomous driving: Challenges and frontiers," *arXiv preprint arXiv:2306.16927*, 2023.